

InDoorLora: Indoor LoRa Positioning System

Comprehensive Technical Report

From Hybrid GNSS + LoRa Architecture to LoRa-Only Indoor
Positioning

Ahmed Al-Muharaq Malick Diop Oumnia Chiouikh
Cynthia Ayetolou

Master 1 – Embedded Computing Systems / IoT Track
Université Marie et Louis-Pasteur – UFR STGI

May 12, 2026

Contents

Executive Summary	6
1 Project Context and Motivation	7
1.1 Why Indoor Positioning Matters	7
1.2 Limitations of GNSS	7
1.3 From GNSS Constraints to Error Correction	8
2 From Hybrid GNSS + LoRa to InDoorLora	10
2.1 Original Hybrid Architecture	10
2.2 LoRa-Only Positioning	10
3 System Architecture	12
3.1 Global Architecture	12
3.2 Main Files	12
3.3 Runtime Communication	13
4 Dashboard and User Interface	14
4.1 Dashboard Objectives	14
4.2 Location Selection and Anchor Configuration	14
4.3 Creating a New Location	15
4.4 MyRoom Live Dashboard	16
4.5 OSM Map Integration	17

5 Database Design	18
5.1 SQLite Tables	18
5.2 Current Database State	18
5.3 Calibration Data	19
6 Positioning Algorithms	20
6.1 RSSI Path-Loss Model	20
6.2 N-Lateration	21
6.3 K-NN Fingerprinting	21
6.4 HMM and Viterbi Smoothing	22
7 Simulator	24
7.1 Purpose of the Simulator	24
7.2 Movement Model	24
7.3 Generated Packet Format	24
8 How to Run the Project	26
8.1 Install Dependencies	26
8.2 Start the Server	26
8.3 Open the Dashboard	26
8.4 Start the Simulator	26
9 Work Completed	28
10 Current Limitations	29
11 Next Steps	30
11.1 Step 1: Clean the Project Structure	30
11.2 Step 2: Rebuild the Database	31

11.3 Step 3: Collect Real Calibration Data	31
11.4 Step 4: Collect Ground-Truth Test Trajectories	32
11.5 Step 5: Evaluate Accuracy	32
11.6 Step 6: Prepare Hardware Integration	33
12 Conclusion	34
A Useful Commands	35
A.1 Run the Project	35
A.2 Export Database Tables	35
A.3 Example WebSocket Actions	35

List of Figures

1.1	Hybrid positioning environments and GNSS limitations: open sky, urban canyon, and indoor/tunnel environments.	8
1.2	GNSS constraints, positioning error definition, and the idea of learning correction patterns through a fingerprinting database.	9
2.1	LoRa-only positioning using geometric N-Lateration and fingerprinting. These methods are used when GNSS is degraded or unavailable.	11
4.1	Dashboard location selector showing two configured locations: M2 Classroom and MyRoom.	14
4.2	New Location window used to define room dimensions, grid step, and anchor positions.	15
4.3	Live dashboard for MyRoom, showing anchors, calibrated points, range circles, RSSI bars, and algorithm outputs.	16
4.4	OSM map view for M2 Classroom, linking indoor coordinates to a geographic reference.	17

List of Tables

11.1 Example structure for algorithm comparison.	33
--	----

Executive Summary

InDoorLora is a real-time indoor positioning prototype based on LoRa radio measurements. The system estimates the position of a user device inside a room using four fixed LoRa anchors named E0, E1, E2, and E3. It receives RSSI measurements, processes them through several positioning algorithms, stores experiment data in an SQLite database, and displays live results on an engineering dashboard.

The project comes from a broader research idea: a hybrid positioning architecture combining GNSS and LoRa for challenging urban and indoor environments. In open sky conditions, GNSS can provide accurate positioning. However, in dense urban canyons, tunnels, and indoor environments, GNSS becomes degraded or unavailable. The current implementation focuses on the most difficult and practical case: GNSS-denied indoor positioning, also called the LoRa-only mode or Case C.

The prototype includes a Python positioning server, a LoRa RSSI simulator, a web dashboard, an SQLite database, and three positioning methods: N-Lateration, K-NN fingerprinting, and HMM/Viterbi smoothing. The current system is already functional as a development prototype. The next major step is to collect a complete real-world calibration dataset, repair or rebuild corrupted logging tables, and evaluate the system using ground-truth trajectories and error metrics.

Chapter 1

Project Context and Motivation

1.1 Why Indoor Positioning Matters

Reliable positioning is essential in many modern applications: emergency response, indoor navigation, asset tracking, smart buildings, industrial monitoring, logistics, and campus services. Outdoor positioning is commonly solved using GNSS systems such as GPS, Galileo, GLONASS, or BeiDou. However, GNSS is not reliable everywhere. It performs well under open sky, but its quality drops significantly in dense urban environments and usually becomes unusable indoors.

Indoor positioning is therefore a real engineering problem. A smartphone, connected object, robot, or tracked asset may need to know its location even when satellite signals are blocked. This is exactly the situation targeted by the InDoorLora project.

1.2 Limitations of GNSS

GNSS positioning depends on the visibility of enough satellites and the quality of satellite geometry. In open sky, many satellites are visible and the receiver can compute a stable position. In urban canyons, buildings block or reflect signals, causing multipath and poor geometry. Indoors or in tunnels, satellite signals may be completely unavailable.

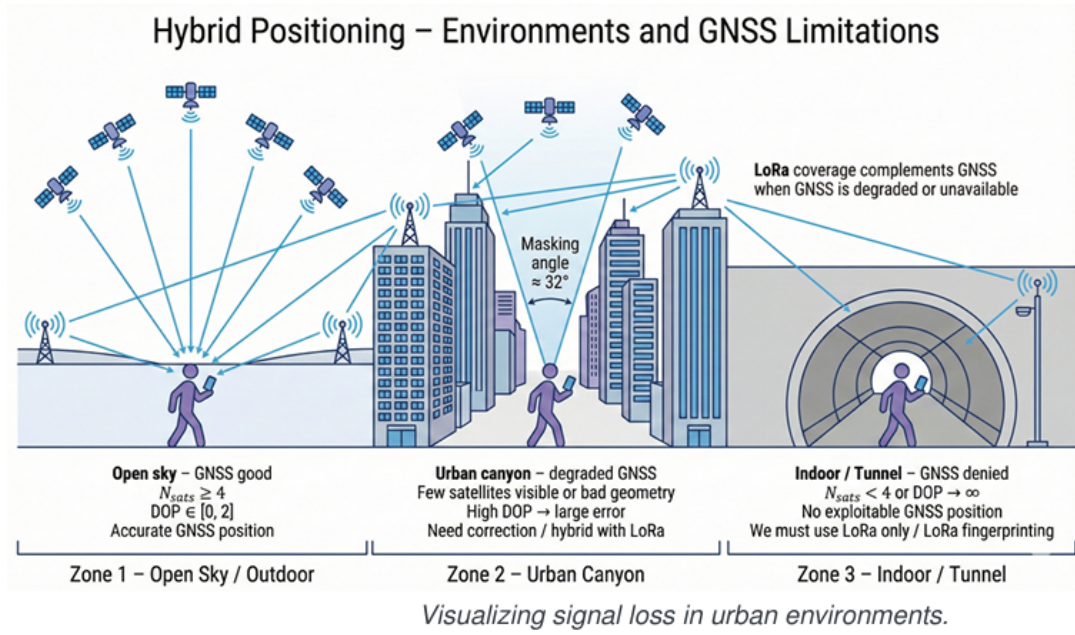
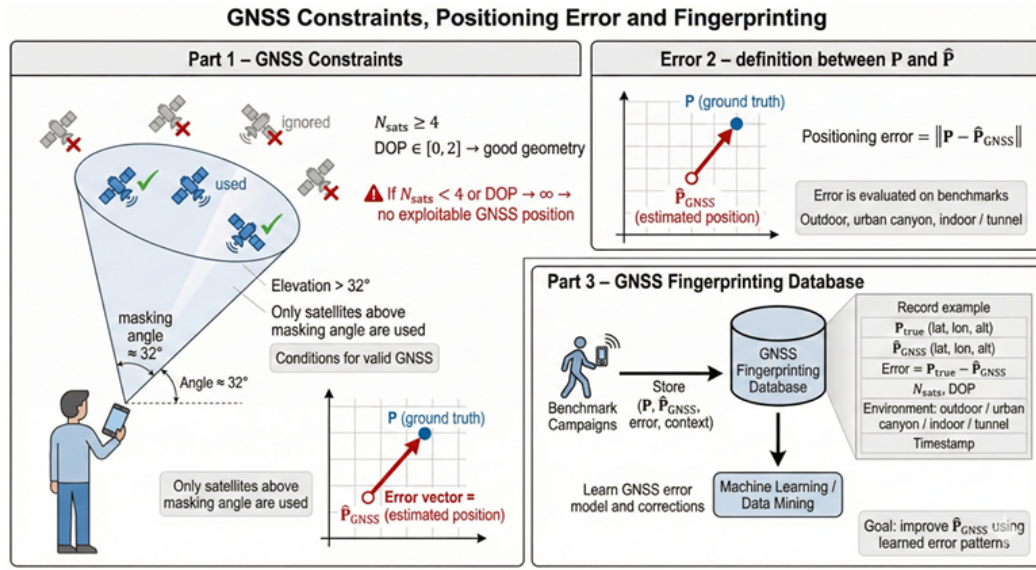


Figure 1.1: Hybrid positioning environments and GNSS limitations: open sky, urban canyon, and indoor/tunnel environments.

Figure 1.1 illustrates the progressive degradation of GNSS. In Zone 1, open-sky GNSS is usable. In Zone 2, urban buildings degrade GNSS by blocking satellites and creating poor geometry. In Zone 3, indoor or tunnel conditions deny GNSS completely, so LoRa-only positioning becomes necessary.

1.3 From GNSS Constraints to Error Correction

The original hybrid project defines validity conditions for GNSS. A GNSS fix is considered reliable only if enough satellites are visible, the satellite elevation is above a masking angle, and the DOP value is acceptable. If these conditions are not satisfied, the system should reject the GNSS position or correct it using LoRa.



Target: Minimize error via Machine Learning.

Figure 1.2: GNSS constraints, positioning error definition, and the idea of learning correction patterns through a fingerprinting database.

The positioning error is computed as the Euclidean distance between the true position and the estimated position:

$$Error = \|P_{true} - \hat{P}\|_2 \quad (1.1)$$

In three dimensions, this becomes:

$$Error = \sqrt{(x - \hat{x})^2 + (y - \hat{y})^2 + (z - \hat{z})^2} \quad (1.2)$$

For the current indoor implementation, the system mainly works in two dimensions:

$$Error_{2D} = \sqrt{(x - \hat{x})^2 + (y - \hat{y})^2} \quad (1.3)$$

The long-term objective is to minimize this error using radio measurements and machine learning methods such as fingerprinting.

Chapter 2

From Hybrid GNSS + LoRa to InDoorLora

2.1 Original Hybrid Architecture

The initial research-oriented project proposed a hybrid positioning strategy based on GNSS and LoRa. The system logic can be divided into three cases:

- **Case A – GNSS good:** GNSS is valid and can be used directly.
- **Case B – GNSS degraded:** GNSS is partially usable but needs correction or reinforcement through LoRa.
- **Case C – GNSS denied:** GNSS is unavailable or unreliable, so the system switches to LoRa-only positioning.

InDoorLora is the current implementation of Case C. This is important: the present prototype is not yet a full GNSS + LoRa fusion engine. It is the indoor LoRa-only positioning layer that will later be integrated into the larger hybrid architecture.

2.2 LoRa-Only Positioning

When GNSS is not available, the system estimates position from LoRa measurements. The main available measurement is RSSI, which represents the received signal strength. In some theoretical cases, ToA or TDoA could also be used, but RSSI is more accessible in the current prototype.

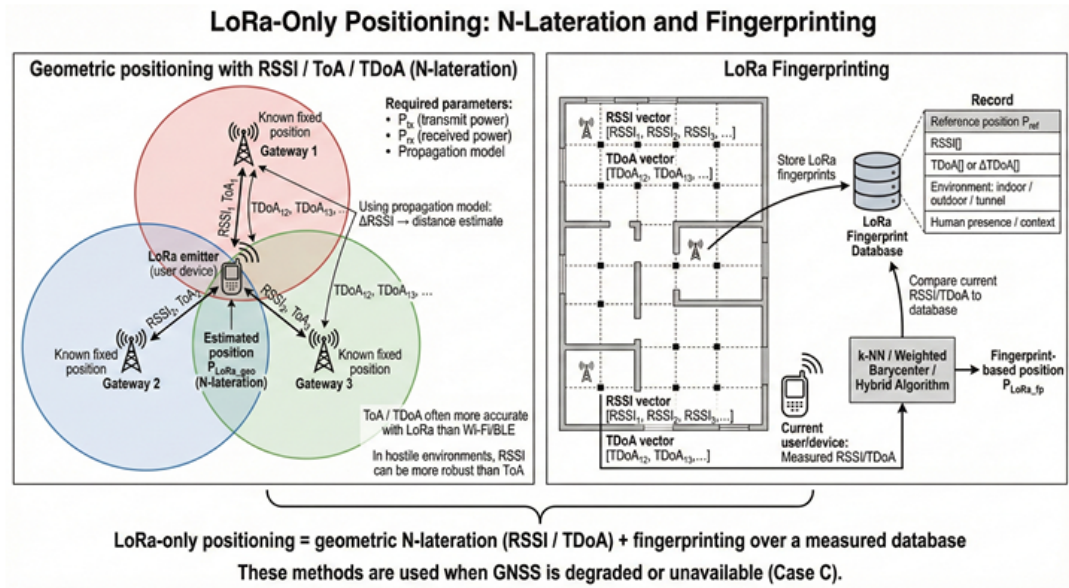


Figure 2.1: LoRa-only positioning using geometric N-Lateration and fingerprinting. These methods are used when GNSS is degraded or unavailable.

Figure 2.1 shows two complementary LoRa-only positioning methods. The first is geometric positioning using RSSI-to-distance estimation and N-Lateration. The second is fingerprinting, where a database of known RSSI patterns is used to estimate the current position.

Chapter 3

System Architecture

3.1 Global Architecture

The current system has three main runtime components:

1. A LoRa receiver source, either a simulator or a real GNU Radio / USRP flowgraph.
2. A Python positioning server.
3. A browser-based dashboard.

The data flow is:

```
lora_rx_sim.py  --ZMQ PUB-->  positioning_server.py  --WebSocket
-->  InDoorLora_Dashboard.html
(or USRP/GRC)      5556              SQLite DB
8765
                                indoorlora.db
```

The simulator or hardware receiver publishes RSSI packets through ZMQ. The server subscribes to these packets, parses them, updates the RSSI buffer, runs positioning algorithms, stores logs in SQLite, and broadcasts position updates to the dashboard using WebSocket.

3.2 Main Files

File	Purpose
<code>positioning_server.py</code>	Main server. It includes ZMQ subscriber, WebSocket server, SQLite management, N-Lateration, K-NN fingerprinting, and HMM Viterbi.
<code>lora_rx_sim.py</code>	Software simulator. It moves a virtual user inside the room and publishes simulated RSSI packets.
<code>InDoorLora_Dashboard.html</code>	Browser dashboard. It displays the room, anchors, grid points, RSSI bars, calibration interface, algorithm outputs, trails, range circles, and OSM map.
<code>indoorlora.db</code>	SQLite database containing locations, anchors, grid points, calibration data, and RSSI logs.
<code>fp.py</code>	Standalone simplified fingerprinting implementation.
<code>hmm.py</code>	Standalone simplified HMM implementation.
<code>nlat.py</code>	Standalone simplified N-Lateration implementation using grid search.
<code>README.md</code>	Project documentation, quick start instructions, WebSocket API, and database description.

3.3 Runtime Communication

The runtime communication uses two protocols:

- **ZMQ:** used for high-frequency packet transport from the LoRa source to the server.
- **WebSocket:** used for real-time interaction between the server and the dashboard.

The ZMQ address is:

```
tcp://127.0.0.1:5556
```

The WebSocket address is:

```
ws://127.0.0.1:8765
```

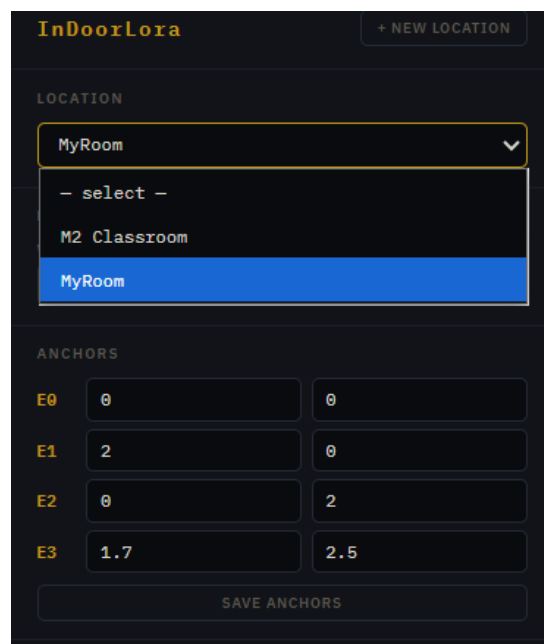
Chapter 4

Dashboard and User Interface

4.1 Dashboard Objectives

The dashboard is designed as an engineering interface, not just a visual map. It allows the user to create rooms, define anchor positions, calibrate grid points, monitor RSSI signals, compare algorithm outputs, and visualize the estimated position in real time.

4.2 Location Selection and Anchor Configuration



The screenshot shows the IndoorLora dashboard interface. At the top, the title 'IndoorLora' is displayed in orange, and a '+ NEW LOCATION' button is in the top right. Below this, the 'LOCATION' section features a dropdown menu currently showing 'MyRoom'. The dropdown is open, revealing a list with '- select -', 'M2 Classroom', and 'MyRoom' (which is highlighted in blue). Below the location selector, the 'ANCHORS' section contains a table of anchor points. The table has two columns for x and y coordinates. The anchors are labeled E0, E1, E2, and E3. At the bottom of the anchors section is a 'SAVE ANCHORS' button.

ANCHORS		
E0	0	0
E1	2	0
E2	0	2
E3	1.7	2.5

Figure 4.1: Dashboard location selector showing two configured locations: M2 Classroom and MyRoom.

Figure 4.1 shows the location selector. The system can manage multiple indoor areas. Each location has its own dimensions, anchors, grid points, and calibration data.

4.3 Creating a New Location

NEW LOCATION		
NAME e.g. Lab B204		
WIDTH (M) 8.0	HEIGHT (M) 6.0	GRID STEP (M) 0.5
ANCHOR POSITIONS		
E0	0.5	0.5
E1	7.5	0.5
E2	0.5	5.5
E3	7.5	5.5
		CANCEL CREATE LOCATION

Figure 4.2: New Location window used to define room dimensions, grid step, and anchor positions.

Figure 4.2 shows the creation of a new indoor location. The user enters a room name, width, height, grid step, and anchor positions. After creation, the server automatically generates grid points and stores the new location in SQLite.

4.4 MyRoom Live Dashboard

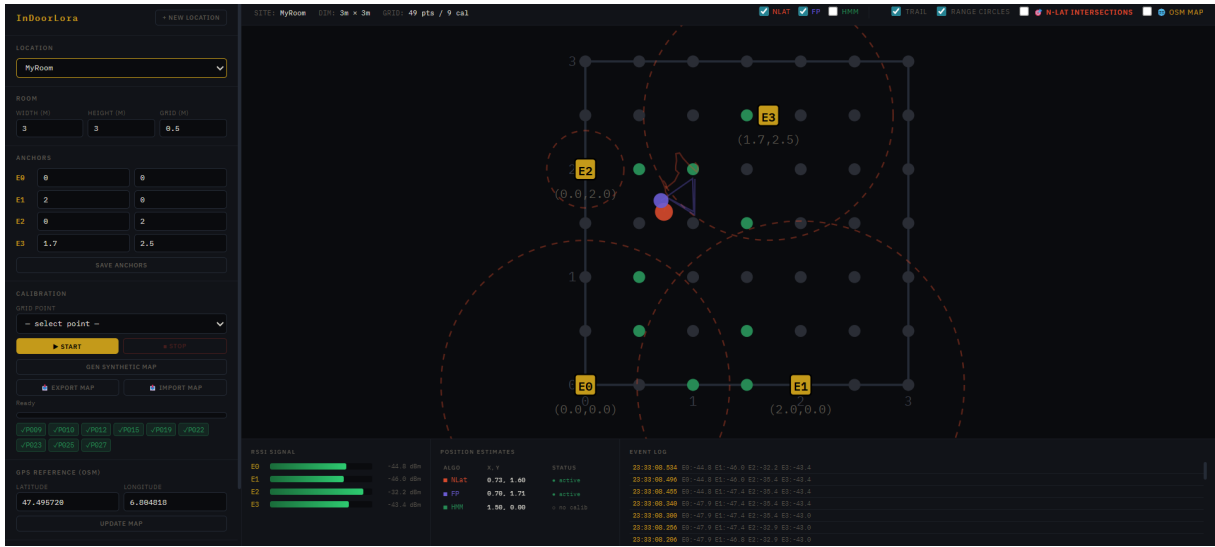


Figure 4.3: Live dashboard for MyRoom, showing anchors, calibrated points, range circles, RSSI bars, and algorithm outputs.

Figure 4.3 shows the live operation of the system in a 3 m by 3 m room. The anchors are placed inside the room, the grid points are displayed, and the active algorithm estimates are shown at the bottom. RSSI values are continuously updated and the event log records incoming measurements.

4.5 OSM Map Integration

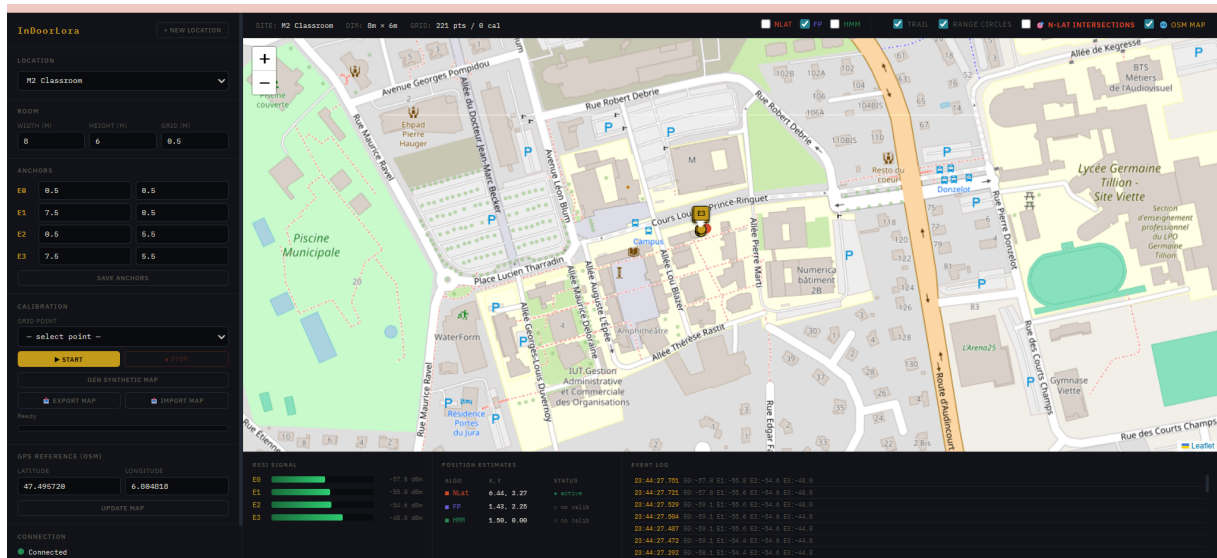


Figure 4.4: OSM map view for M2 Classroom, linking indoor coordinates to a geographic reference.

Figure 4.4 shows the OSM integration. The user can provide a GPS reference coordinate, and the indoor coordinate system is projected onto an outdoor map. This does not mean GNSS is used for indoor positioning. It is mainly a visualization and geographic anchoring feature.

Chapter 5

Database Design

5.1 SQLite Tables

The database stores all persistent information required by the system. The main tables are:

Table	Description
<code>locations</code>	Stores room name, width, height, grid step, creation time, and active status.
<code>anchors</code>	Stores anchor coordinates for each location.
<code>grid_points</code>	Stores automatically generated calibration points for each location.
<code>calibration_data</code>	Stores raw RSSI samples collected during calibration for each grid point and anchor.
<code>rss_i_log</code>	Stores live processed RSSI readings and estimated positions for each algorithm.

5.2 Current Database State

The inspected database contains two locations:

- **M2 Classroom:** 8 m by 6 m, grid step 0.5 m.
- **MyRoom:** 3 m by 3 m, grid step 0.5 m.

The project currently includes several anchors, generated grid points, and a small amount of calibration data. However, the RSSI log table appears to contain corruption. Therefore, for final experiments, the safest decision is to back up the database, rebuild the corrupted table, and collect a clean dataset.

5.3 Calibration Data

Calibration is the process of recording RSSI values at known positions. For each grid point, the system collects a fixed number of packets from all anchors. The default target is 60 samples per point. After calibration, the point becomes part of the radio map and can be used by fingerprinting and HMM.

Chapter 6

Positioning Algorithms

6.1 RSSI Path-Loss Model

The geometric part of the system uses a path-loss model to convert RSSI into distance:

$$RSSI(d) = A - 10n \log_{10}(d) \quad (6.1)$$

where:

- A is the expected RSSI at 1 meter.
- n is the path-loss exponent.
- d is the estimated distance.

The inverted formula is:

$$d = 10^{\frac{A - RSSI}{10n}} \quad (6.2)$$

In the current server implementation, values close to $A = -42$ dBm and $n = 2.2$ are used. These values are acceptable for simulation, but real-world experiments require calibration because RSSI changes with walls, antenna orientation, human body loss, multipath, and interference.

6.2 N-Lateration

N-Lateration estimates the user position using distances to fixed anchors. Each anchor defines a circle around itself. The user should theoretically be at the intersection of these circles. In real indoor conditions, the circles do not intersect perfectly because RSSI is noisy. Therefore, the algorithm solves an optimization problem:

$$\hat{P} = \arg \min_P \sum_{i=1}^N (\|P - A_i\| - d_i)^2 \quad (6.3)$$

where A_i is the position of anchor i , and d_i is the distance estimated from RSSI.

Advantages:

- Does not require a calibration database.
- Can work immediately after anchor placement.
- Useful as a baseline method.

Limitations:

- Strongly depends on the correctness of the RSSI-to-distance model.
- Sensitive to multipath and body shadowing.
- Less reliable in complex indoor environments.

6.3 K-NN Fingerprinting

Fingerprinting uses a radio map instead of converting RSSI to distance. During calibration, each grid point stores a vector such as:

$$F_i = [RSSI_{E0}, RSSI_{E1}, RSSI_{E2}, RSSI_{E3}] \quad (6.4)$$

During live operation, the current vector is compared against the stored fingerprints:

$$V = [r_0, r_1, r_2, r_3] \quad (6.5)$$

The K nearest fingerprints are selected based on signal distance. The position is then estimated using weighted averaging:

$$\hat{P} = \frac{\sum_{i=1}^K w_i P_i}{\sum_{i=1}^K w_i} \quad (6.6)$$

where the weight can be computed as:

$$w_i = \frac{1}{d_i + \epsilon} \quad (6.7)$$

Advantages:

- Usually more robust indoors than pure N-Location.
- Learns the real radio behavior of the room.
- Can account indirectly for walls and multipath.

Limitations:

- Requires calibration work.
- The radio map may become outdated if the environment changes.
- Accuracy depends on grid density and data quality.

6.4 HMM and Viterbi Smoothing

The HMM method models each grid point as a hidden state. The RSSI vector is the observation. The algorithm combines two ideas:

- **Emission probability:** how likely the observed RSSI vector is for a given grid point.
- **Transition probability:** how likely the user is to move from one grid point to another between two time steps.

This method is useful because real users cannot teleport across the room. If one RSSI sample suddenly suggests a far-away position, the HMM can reduce the effect of this outlier.

Advantages:

- Produces smoother trajectories.
- Reduces random jumps.
- Uses temporal consistency.

Limitations:

- Requires a calibrated radio map.
- Needs good tuning of transition and observation parameters.
- Can lag behind fast movement if too much smoothing is applied.

Chapter 7

Simulator

7.1 Purpose of the Simulator

The simulator allows the full system to be tested without physical LoRa hardware. It creates a virtual moving user and generates RSSI values for each anchor according to the distance between the virtual user and the anchor.

7.2 Movement Model

The simulated user follows a rectangular path inside the active room. The path keeps a margin from the walls. The simulator adapts automatically when the active location changes because it receives room dimensions and anchor positions through WebSocket.

7.3 Generated Packet Format

The simulator sends a PDU format compatible with the server:

```
[4-byte BE meta_len][meta JSON][4-byte BE payload_len][payload
  string]
```

Example metadata:

```
{"rssi": -42.1, "samp_rate": 250000}
```

Example payload:

UE07 , 13 , E0 , seq

This design makes it possible to replace the simulator with a GNU Radio LoRa receiver later, as long as the same PDU format is respected.

Chapter 8

How to Run the Project

8.1 Install Dependencies

```
pip install websockets pyzmq scipy numpy websocket-client
```

8.2 Start the Server

```
python positioning_server.py
```

At first launch, the server creates the SQLite database if it does not already exist and seeds a default location.

8.3 Open the Dashboard

Open the following file in a browser:

```
InDoorLora_Dashboard.html
```

Chrome or Firefox is recommended.

8.4 Start the Simulator

```
python lora_rx_sim.py
```

After this, the dashboard should start receiving RSSI values and displaying position estimates.

Chapter 9

Work Completed

The work completed so far includes:

- Designing a research-oriented hybrid GNSS + LoRa architecture.
- Identifying the three operating modes: GNSS good, GNSS degraded, and GNSS denied.
- Implementing the LoRa-only indoor positioning layer.
- Building a Python positioning server using ZMQ, WebSocket, and SQLite.
- Implementing N-Lateration based on RSSI path-loss inversion.
- Implementing K-NN fingerprinting using a calibration radio map.
- Implementing HMM/Viterbi smoothing for temporal consistency.
- Building a live dashboard for visualization, calibration, and monitoring.
- Adding support for multiple locations.
- Adding a New Location window for custom room dimensions and anchor positions.
- Adding grid point generation.
- Adding a calibration workflow.
- Adding radio map import and export.
- Adding range circles and N-Lateration intersection visualization.
- Adding OSM map integration.
- Building a dynamic simulator that adapts to the active room.

Chapter 10

Current Limitations

The current prototype is functional, but it still has limitations:

1. The current database contains a corrupted RSSI log table, so final experiments should use a clean database.
2. The available CSV datasets are mostly empty and cannot yet support a serious performance evaluation.
3. The amount of calibration data is still small.
4. There is no complete ground-truth trajectory dataset.
5. The current path-loss parameters are fixed and should be calibrated for the real room.
6. N-Lateration is sensitive to RSSI noise and should be treated as a baseline.
7. Fingerprinting and HMM require a complete calibration map.
8. The simulator validates the software pipeline but does not prove real-world radio accuracy.
9. The algorithm code exists both inside the server and in standalone files, so the final project structure should be cleaned.

These limitations are normal for a prototype stage. The most important missing element is not the software architecture; it is the experimental dataset and evaluation.

Chapter 11

Next Steps

11.1 Step 1: Clean the Project Structure

The project should be reorganized into a clean structure:

```
InDoorLora/  
  server/  
    positioning_server.py  
  dashboard/  
    InDoorLora_Dashboard.html  
  simulator/  
    lora_rx_sim.py  
  algorithms/  
    nlat.py  
    fp.py  
    hmm.py  
  data/  
    indoorlora.db  
    exports/  
  docs/  
    report.pdf  
    figures/  
  README.md  
  requirements.txt
```

A requirements.txt file should be added:

```
websockets  
pyzmq
```

```
scipy  
numpy  
websocket-client
```

11.2 Step 2: Rebuild the Database

Because the current `rss_i_log` table appears corrupted, the safest next step is:

1. Backup the existing database.
2. Export useful calibration data if possible.
3. Create a clean SQLite database.
4. Recreate locations and anchors.
5. Run new calibration and logging experiments.

11.3 Step 3: Collect Real Calibration Data

For a strong final report, calibration must be systematic:

1. Select one test room, for example M2 Classroom.
2. Measure room dimensions precisely.
3. Place E0, E1, E2, and E3 at fixed known coordinates.
4. Generate grid points with a step of 0.5 m or 1 m.
5. At each grid point, collect at least 60 to 120 RSSI samples.
6. Repeat measurements with different user orientations if possible.
7. Save all data in `calibration_data`.

11.4 Step 4: Collect Ground-Truth Test Trajectories

Calibration data is not enough. A separate test dataset is needed. The user should walk along predefined paths with known coordinates. For example:

- Straight line path.
- Rectangular path.
- Diagonal path.
- Stop-and-go path.

For every test point, the system should store:

- timestamp,
- true x and y position,
- RSSI from each anchor,
- estimated position from NLat,
- estimated position from FP,
- estimated position from HMM.

11.5 Step 5: Evaluate Accuracy

The evaluation should compare algorithms using:

- Mean Absolute Error,
- Root Mean Square Error,
- Median error,
- 90th percentile error,
- CDF error curve,
- trajectory plots.

The expected final comparison table should look like this:

Table 11.1: Example structure for algorithm comparison.

Algorithm	Mean Error	RMSE	Median Error	90th Percentile
N-Lateration	—	—	—	—
K-NN Fingerprinting	—	—	—	—
HMM Viterbi	—	—	—	—

The values are intentionally left empty because they must come from real experiments.

11.6 Step 6: Prepare Hardware Integration

The simulator should eventually be replaced by real LoRa reception. The planned hardware path is:

1. Configure GNU Radio with LoRa receiver blocks.
2. Extract RSSI and anchor identifier from received packets.
3. Publish packets through a ZMQ PUB sink at `tcp://127.0.0.1:5556`.
4. Keep the same PDU format used by the simulator.
5. Validate that the server receives and processes real packets.

Chapter 12

Conclusion

InDoorLora is a strong practical step toward robust positioning in GNSS-denied environments. The project started from a broad hybrid GNSS + LoRa architecture and has now produced a working indoor LoRa-only prototype. The system includes real-time data transport, database persistence, live visualization, room configuration, calibration support, and three complementary positioning algorithms.

The most important achievement is that the project is no longer only theoretical. It has an executable pipeline from radio measurement generation to position estimation and dashboard visualization. However, the project is not finished. The next phase must focus on data quality and scientific evaluation: clean database, complete calibration, ground-truth trajectories, and quantitative error analysis.

The final version of the project should clearly present InDoorLora as the Case C implementation of the larger Hybrid GNSS + LoRa positioning system. This makes the work coherent: GNSS is used when available, hybrid correction is used when degraded, and InDoorLora handles the indoor case where GNSS is denied.

Appendix A

Useful Commands

A.1 Run the Project

```
pip install websockets pyzmq scipy numpy websocket-client
python positioning_server.py
python lora_rx_sim.py
```

A.2 Export Database Tables

```
sqlite3 indoorlora.db ".headers on" ".mode csv" "SELECT * FROM
    rssi_log" > rssi_log.csv
sqlite3 indoorlora.db ".headers on" ".mode csv" "SELECT * FROM
    calibration_data" > calib.csv
```

A.3 Example WebSocket Actions

```
{"action": "get_locations"}
{"action": "set_active", "location_id": 1}
{"action": "start_calibration", "grid_point_id": 42}
{"action": "stop_calibration"}
```